AI MONEY GROUP

PRACTICAL AI SYSTEMS

THE COMPLETE STARTER SYSTEM

CLAUDE CODE SMART ORCHESTRATOR KIT

Route planning, difficult reasoning, implementation, and review to the right agent without wasting context.

4 AGENT TEMPLATES | ROUTING MATRIX | SETUP + TESTS

COPY • INSTALL • TEST • CUSTOMIZE

Start here

Claude Code becomes much more useful when one conversation stops trying to do every job.

A difficult project usually contains several kinds of work:

- understanding the current system
- making architecture decisions
- investigating uncertain failures
- implementing routine changes
- running tests and reviewing the result

Those jobs do not need the same instructions, tool access, or amount of reasoning. When one agent carries the entire project, its context fills with file searches, logs, failed experiments, boilerplate, and repeated explanations. The result can be slower, harder to audit, and more expensive than necessary.

This kit gives you a simple alternative:

1. Keep one main conversation in charge.
2. Classify the work before delegating.
3. Send bounded tasks to specialized agents.
4. Require evidence at every handoff.
5. Verify the finished work independently.

> **The main idea:** Do not route work by hype or by whichever model is newest. Route it by the kind of judgment the task requires.

What is included

- A ready-to-copy project `CLAUDE.md`
- Four specialized Claude Code subagents
- A task-routing matrix
- Installation instructions
- Test prompts
- A visual workflow map
- Cost and context controls
- Troubleshooting guidance
- A downloadable starter bundle

You can install the starter files as written, then adapt the tools and instructions to your project.

The problem with one-agent-everything

A single Claude Code conversation can plan, search, edit, test, and review. That does not mean it should perform all five jobs in one undifferentiated loop.

Context gets polluted

Search output, stack traces, build logs, and exploratory dead ends compete with the requirements that actually matter. When the important instructions become a small part of a crowded context window, the agent has more opportunities to miss them.

Routine work receives too much attention

Boilerplate, formatting, test fixture updates, and repetitive refactors usually need clear acceptance criteria more than deep architectural reasoning. Spending the most capable reasoning process on mechanical edits adds overhead without guaranteeing a better result.

Completion becomes self-reported

The same agent that wrote a change is tempted to accept its own explanation. It may say the work is complete because the code looks plausible, even when a requirement was missed or a test was never run.

Parallel work can collide

Two agents editing the same file can overwrite each other, duplicate work, or return incompatible versions. Parallelism only helps when the tasks are independent and the ownership boundaries are clear.

Delegation can become theater

More agents do not automatically produce a better system. If each worker receives vague instructions, repeats the same research, or returns a long narrative instead of a usable result, the orchestrator has created more context, not less.

The Smart Orchestrator solves these problems with roles, boundaries, and verification gates.

The architecture

The system has five stages:

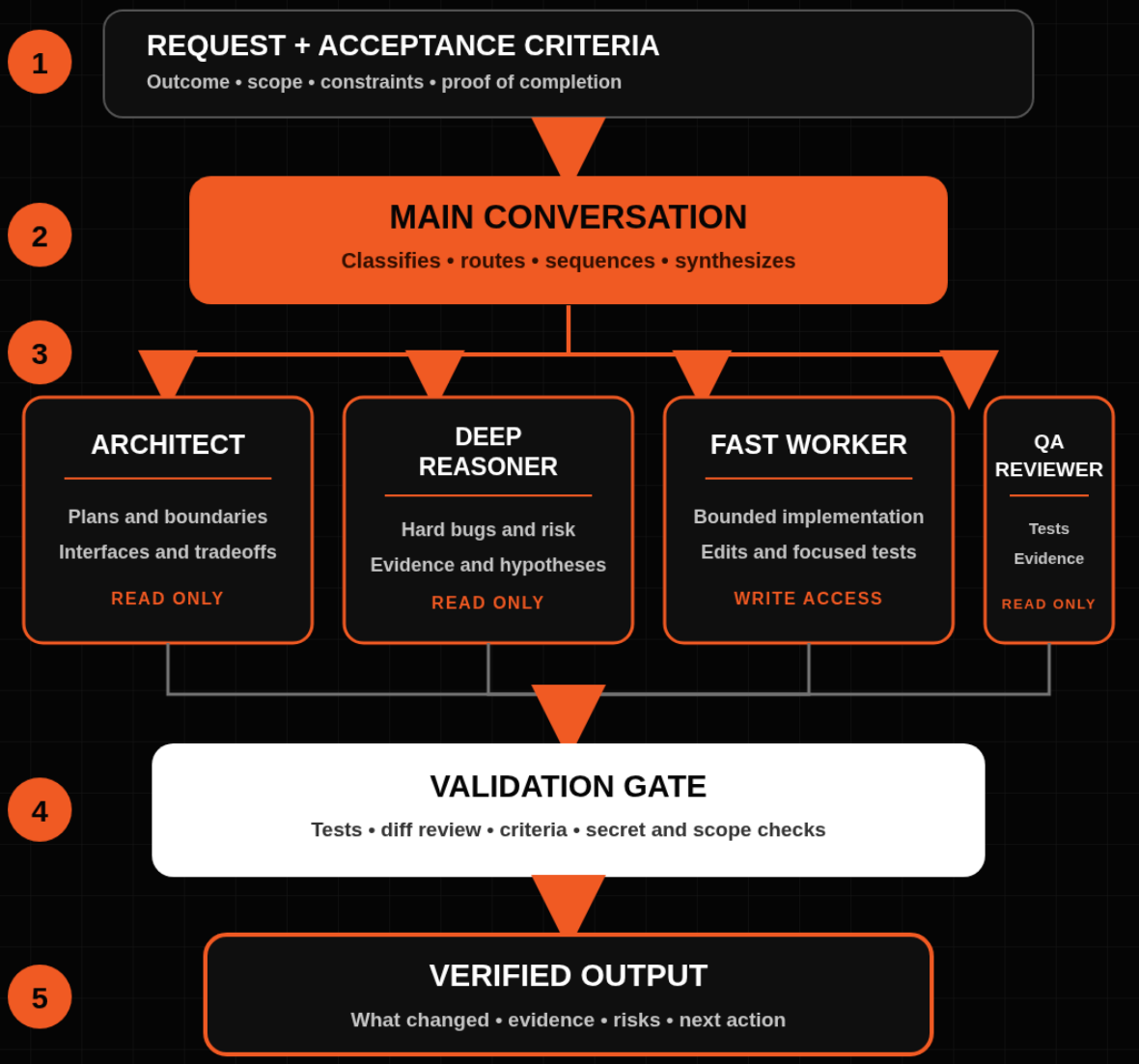
1. **Request intake:** Define the desired outcome, constraints, and success criteria.
2. **Orchestration:** The main conversation classifies the work and owns scope, sequence, and synthesis.
3. **Role routing:** Specialized agents handle architecture, difficult reasoning, implementation, or review.
4. **Validation gate:** Tests, file inspection, and criterion-by-criterion review determine whether the work passes.
5. **Final output:** The main conversation reports what changed, what was verified, and what remains uncertain.

AI MONEY GROUP

PRACTICAL AI SYSTEMS

CLAUDE CODE SMART ORCHESTRATOR

Route the right work to the right agent.



ONE ORCHESTRATOR. BOUNDED TASKS. VERIFIED RESULTS.

Important: The agents can reason about the next action. Your workflow should still use explicit rules for scope, permissions, file ownership, test commands, and approval gates. Reasoning is not a substitute for deterministic controls.

Meet the four agents

1. Architect

The Architect is read-only. It inspects the repository and turns a broad request into an implementation-ready plan.

Use it for:

- multi-file features
- interface or data-flow changes
- architecture decisions
- migration planning
- dependency ordering
- high-risk work where the wrong direction is expensive

It should return exact file paths, tradeoffs, ordered tasks, acceptance criteria, and verification commands. It should not write code.

Do not use it for

- a one-line copy change
- obvious formatting
- repetitive edits with a clear pattern
- work that already has a complete implementation plan

2. Deep Reasoner

The Deep Reasoner is also read-only. It is reserved for uncertain problems where evidence and competing hypotheses matter.

Use it for:

- difficult bugs
- ambiguous root causes

- security-sensitive decisions
- complex algorithms
- unexpected performance behavior
- decisions with expensive consequences

It should gather a small evidence set, test hypotheses, state its confidence, and recommend the smallest corrective action.

Do not use it for

- routine implementation
- broad codebase tours
- mechanical file updates
- generating long explanations when a test can answer the question

3. Fast Worker

The Fast Worker implements a clear, bounded assignment. It can read, edit, write, and run commands.

Use it for:

- boilerplate
- focused refactors
- test updates
- formatting
- routine code changes
- implementing one task from an approved plan

It should touch only the allowed files, follow existing patterns, run the specified verification, and report the exact files and command results.

Do not use it when

- the architecture is unresolved
- requirements conflict
- the assignment does not name acceptance criteria
- another agent is editing the same files

4. QA Reviewer

The QA Reviewer independently checks the result. It does not edit source files and should never approve work based only on another agent's summary. It has Bash access so it can run approved checks; those commands may still create caches or build artifacts, so test it in a disposable branch and review commands before approval.

Use it for:

- requirement verification
- final diff review
- targeted tests and builds
- security and secret checks
- regression checks
- scope-control review

It returns one of three verdicts:

- **PASS:** Every acceptance criterion is verified.
- **PASS WITH WARNINGS:** Requirements pass, but non-blocking risks remain.
- **FAIL:** One or more criteria are not met.

Install the starter system

Step 1: Copy the files

Place `CLAUDE.md` and the `.claude` directory in the root of your Claude Code project.

```
project-root/
├── CLAUDE.md
└── .claude/
    └── agents/
        ├── architect.md
        ├── deep-reasoner.md
        ├── fast-worker.md
        └── qa-reviewer.md
```

Project subagents in `.claude/agents/` can be checked into version control so the team shares the same roles and constraints.

Step 2: Start Claude Code

```
cd path/to/project
claude
```

If the `.claude/agents/` directory did not exist when your session started, restart Claude Code after creating it.

Step 3: Confirm discovery

Ask:

```
Which project subagents are available, and what is each one for?
```

You should see the four agent names and descriptions.

Step 4: Run a read-only test

```
Use the architect agent to inspect this project and propose one small improvement. Do not edit files.
```

Confirm that the Architect returns a plan without changing the project.

Step 5: Test the full sequence safely

Use a disposable branch or sample project.

```
For this small change, use architect first, fast-worker second, and qa-reviewer last. Keep each assignment bounded. Do not let agents delegate further. Report the evidence from each stage.
```

Setup tip: Keep `model: inherit` in the public starter. It makes the files portable. If you choose different models for different roles, make that decision in your private project based on your plan, provider, risk, and current availability.

The routing matrix

Work type	First route	Next route	Verification
Small, obvious edit	Fast Worker	None	QA Reviewer
Multi-file feature	Architect	Fast Worker	QA Reviewer
Difficult bug	Deep Reasoner	Fast Worker	QA Reviewer
Architecture decision	Architect	Deep Reasoner only if risk is high	QA Reviewer
Boilerplate or repetitive refactor	Fast Worker	None	QA Reviewer
Security-sensitive change	Architect	Deep Reasoner, then Fast Worker	QA Reviewer
Read-only compliance audit	QA Reviewer	Architect if redesign is needed	Main conversation
High-volume logs or test output	Deep Reasoner or QA Reviewer	Fast Worker if a fix is clear	QA Reviewer

A faster decision rule

Ask three questions:

1. Is the task already clear and bounded?
2. Would a wrong decision be expensive, dangerous, or hard to reverse?
3. Does the work need file edits or independent verification?

Route the task this way:

- Clear, low-risk, editable work goes to the Fast Worker.
- Unclear, high-consequence reasoning goes to the Architect or Deep Reasoner.
- Finished work goes to the QA Reviewer.

The delegation packet

An agent should never receive only a topic. It needs a bounded assignment.

Use this packet every time:

```
OBJECTIVE
What outcome must this task produce?

INPUTS
Which files, errors, requirements, or prior decisions matter?

ALLOWED SCOPE
Which files or components may the agent inspect or modify?

EXCLUDED SCOPE
What must remain untouched?

OUTPUT
What should the agent return: plan, diagnosis, code, test evidence, or verdict?

ACCEPTANCE CRITERIA
What observable conditions must be true when the task is done?

VERIFICATION
Which command or inspection proves the result?
```

Weak delegation

```
Fix the authentication system.
```

The task is broad, the scope is unclear, and completion cannot be measured.

Strong delegation

```
OBJECTIVE
Make expired password-reset tokens return HTTP 410 instead of HTTP 500.

INPUTS
The failing request is covered by tests/auth/test_password_reset.py.
Inspect the reset handler and token validation path.

ALLOWED SCOPE
src/auth/reset.py
tests/auth/test_password_reset.py

EXCLUDED SCOPE
Do not change token lifetime, email templates, or database schema.

OUTPUT
Implement the smallest fix and report files changed.

ACCEPTANCE CRITERIA
The expired-token test returns 410.
Existing password-reset tests still pass.

VERIFICATION
pytest tests/auth/test_password_reset.py -v
```

The second assignment gives the worker a finish line.

Three complete orchestration examples

Example 1: Add a small feature

Request: Add a health-check endpoint.

Route

1. Architect inspects routing, conventions, and existing tests.
2. Main conversation converts the plan into one bounded implementation task.
3. Fast Worker adds the endpoint and tests.
4. QA Reviewer runs the focused test and checks the diff.

Why this route works

The Architect prevents the worker from inventing a new pattern. The Fast Worker handles the routine code. The QA Reviewer confirms that the endpoint matches the request and did not introduce unrelated changes.

Example 2: Diagnose a slow test

Request: Find out why one test takes much longer than the rest.

Route

1. Deep Reasoner measures the test and gathers evidence.
2. It returns competing hypotheses and the most likely root cause.
3. Fast Worker implements one narrow fix.
4. QA Reviewer reruns the test and compares the result.

Why this route works

The reasoning agent does not mix diagnosis with implementation. The worker receives a tested hypothesis instead of guessing. The reviewer checks the measured result rather than accepting a promise that performance improved.

Example 3: Change an authorization rule

Request: Allow managers to export reports while preserving tenant boundaries.

Route

1. Architect maps authorization checks, data flow, and affected tests.
2. Deep Reasoner evaluates cross-tenant and privilege-escalation risks.
3. Fast Worker implements the approved rule in named files.
4. QA Reviewer runs authorization tests and searches for bypass paths.

Why this route works

A security-sensitive change needs more than fast code generation. The planning and reasoning stages reduce the chance that a local fix creates a system-wide vulnerability.

Cost and context controls

The kit cannot promise a fixed percentage of savings. Usage depends on the task, project, model configuration, prompt size, test output, retries, and provider plan. It can help you avoid unnecessary work.

1. Use deep reasoning by exception

Reserve the Deep Reasoner for ambiguity, consequence, or risk. Do not send routine formatting and boilerplate through the same reasoning path.

2. Keep assignments small

A bounded task is cheaper to understand, easier to verify, and safer to retry. If a worker needs half the repository to make one edit, the assignment is not ready.

3. Return summaries, not transcripts

Subagents have isolated contexts. Ask them to return decisions, evidence, paths, and test results. Do not flood the main conversation with every search result and log line.

4. Limit tool access

Read-only roles do not need Edit or Write. The Architect and Deep Reasoner also omit Bash and use plan permission mode. The QA Reviewer keeps Bash only so it can run approved checks; tests can still create caches or build artifacts. Workers do not need the Agent tool. Tool allowlists, permission modes, operator review, and sandboxing provide stronger boundaries than instructions alone.

5. Prevent duplicate exploration

Give each agent the relevant findings from the prior stage. Do not ask four agents to independently rediscover the same repository structure unless comparison is the actual goal.

6. Avoid parallel edits to shared files

Parallelize independent research or separate components. Serialize changes that touch the same files or interfaces.

7. Define stop conditions

Tell the agent when to stop:

- requirements conflict
- a required file does not exist
- a test cannot be run
- the task requires credentials
- the change exceeds allowed scope
- evidence contradicts the proposed approach

A clean blocker report is better than a confident guess.

Deterministic control versus agent judgment

Agents are useful because they can interpret messy tasks. That flexibility also makes them nondeterministic.

Use agent judgment for:

- choosing which files deserve inspection
- comparing design tradeoffs
- forming debugging hypotheses
- explaining evidence
- adapting a bounded implementation to existing patterns

Use explicit controls for:

- which files may be edited
- which commands may run
- which agent owns a task
- whether agents may delegate
- required test commands
- approval criteria
- secret and permission boundaries
- the order of high-risk steps

Operator rule: Let agents reason inside a box. Define the box yourself.

The validation gate

A strong workflow treats verification as a separate deliverable.

Criterion-by-criterion review

The QA Reviewer should map every acceptance criterion to evidence:

Criterion	Evidence	Status
Required file changed	Diff and file path	Pass or fail
Requested behavior works	Focused test output	Pass or fail
Existing behavior remains intact	Relevant regression test	Pass or warning
No unrelated scope	Final diff inspection	Pass or fail
No secrets introduced	Pattern search and diff review	Pass or fail

Evidence hierarchy

Prefer stronger evidence:

1. A passing automated test
2. A successful build or lint command
3. Direct inspection of the final artifact
4. A reproducible manual check
5. An agent's explanation

The explanation comes last because it is not proof by itself.

Common failure modes

The orchestrator keeps doing all the work

Cause: Agent descriptions are vague, or the prompt never asks for explicit delegation.

Fix: Name the agent in the request or use an agent mention. Make descriptions state exactly when the role should be used.

The agents are not discovered

Cause: The `.claude/agents/` directory was created after the session started, the YAML is invalid, or agent names collide.

Fix: Restart Claude Code if the directory is new. Validate the frontmatter. Keep every `name` unique.

The worker edits unrelated files

Cause: The assignment lacks allowed and excluded scope.

Fix: Name the permitted files and state what must remain untouched. Review the final diff.

The reviewer approves without running tests

Cause: The verification command was not part of the assignment.

Fix: Put the exact command in the delegation packet and require the reviewer to report its outcome.

Agents keep spawning more agents

Cause: Their tool list includes the Agent tool.

Fix: Omit Agent from the subagent's `tools` list. Keep orchestration in the main conversation.

Two workers overwrite each other

Cause: Parallel tasks share file ownership.

Fix: Split by independent files or serialize the work. The orchestrator should own the edit map.

The workflow uses more context than before

Cause: Tasks are duplicated, agent returns are too long, or every request goes through every role.

Fix: Route by need. A small change may require only the Fast Worker and QA Reviewer.

Your 30-minute implementation sprint

Minutes 1 to 5: Install

Copy the starter `CLAUDE.md` and `.claude/agents/` directory into a sample project.

Minutes 6 to 10: Validate

Run the included validator and ask Claude Code to list the available agents.

Minutes 11 to 15: Test the Architect

Ask for a read-only plan for one small improvement.

Minutes 16 to 22: Test the Worker

Give the Fast Worker one harmless, bounded documentation change with a clear verification step.

Minutes 23 to 27: Test the Reviewer

Have the QA Reviewer inspect the change and return a verdict with evidence.

Minutes 28 to 30: Customize

Add your real project commands, file boundaries, and completion gates to `CLAUDE.md`.

Do not start with your highest-risk repository. Learn the routing behavior on a small project first.

Turn the system into a service

The value is not the four Markdown files. The value is helping a business define the right roles, permissions, handoffs, and verification for its actual work.

A consultant can use this system to deliver:

- a repository-specific agent map
- project instructions and tool permissions
- repeatable development workflows
- test and review gates
- context and usage controls
- team onboarding documentation

Start with one painful workflow, measure the before and after, and improve it. Do not promise savings or speed you have not measured.

Want help implementing systems like this?

AI Money Group helps entrepreneurs and operators turn AI tools into practical workflows, offers, and income-producing systems.

Visit AI Money Group

For consultants and agencies that want a professional methodology for delivering AI systems to clients, explore **AI Certified Consultant**.

Explore AI Certified Consultant

Official resources

Claude Code custom subagents: **Official custom subagents documentation**

Claude Code project instructions and memory: **Official project instructions and memory documentation**

Claude Code settings: **Official settings documentation**

Claude Code permissions: **Official permissions documentation**

Responsible use

This kit is educational. Tool behavior, model availability, permissions, and provider plans can change. Review the current official documentation before using the templates in sensitive or production environments.

Test the system in a disposable branch or sample project. Review commands before approving them. Never place credentials or private data in prompts, agent files, screenshots, or shared logs.

Results depend on the project, requirements, configuration, team, and verification process. The kit does not guarantee lower cost, faster delivery, or error-free output.